AI Report

DD2 Project 1, Option 1: From a Boolean Function to an HD Cell. CSCE 3304, Fall 2026, Group 5 (`dd2_g5__`).

Members	Hassan Ashraf, Yara Abdelkader and Youssef Ayman Hassan
Way of using AI	AI as a development aid. No AI model runs when the tool runs: the same request always gives the same cell.
Evidence	`AGENTS.md` (read by the assistant as `CLAUDE.md`), `ai/use_log.md` (23 rows on `main`, U1 to U25), `ai/mistakes_log.md` (31 rows, M1 to M31). Every number below is read from these files or from the files under `results/` they cite.

1. What the AI did and what we did, part by part

Every logged AI session was Claude Code in the repository, under the contract in `AGENTS.md`; the use log's Who column names the member whose part each session worked on. That member stated the goal, took the decisions the assistant asked for, and judged the results from the check scripts; the assistant read the handouts and code, wrote and edited the code, ran the `make` targets and wrote the log rows. The table lists each part of the tool, what the AI produced, what the group decided or did itself, and the log rows behind it.

Part	What the AI did	What the group did	Log
Repository and checks	Restructured the repository into `src/`, `tests/`, `cells/`, `results/`; wrote `rules.json` and `pdk_rules.py`, the Makefile targets and broken cases; made the switch, transistor and model checks read the function from the request's truth table.	Wrote and agreed `AGENTS.md`; asked explicitly for every change to a check script (AGENTS.md rule 9).	U1
Synthesis	Quine-McCluskey, factoring into stages of at most 3 series transistors, the stage dynamic program, Ashenhurst decomposition, reference cells read from the installed SPICE netlist.	Chose the objective (fewest transistors) and the Table 6 function of our choice ($A \oplus B \oplus C$).	U2, U7
Transistor ordering	Exact dual-trail Euler search after Maziasz and Hayes, stage placement, folding into fingers, unsplit covers.	Named the method and asked for it to be cited in the code; asked for the folded fingers to share poly.	U3, U10, U18
Layout planner and router	Gate-pitch dynamic program, A* li1/met1 router, rip-up and reroute, the negotiated (PathFinder) router, contact stacking, parallel cuts, pin access.	Set the goals (narrower cells, li1 before met1, stacked contacts, more cuts, pin access); reviewed layout pictures and caught two faults no check covers (M21, M25); had the pin patches removed again.	U3, U4, U10–U12, U15–U18, U20
Characterizer	Magic extraction with SA/SB on every transistor, the slowest side input, input capacitance, the Liberty writer; the Table 5 validation and the SPM design-level run.	Installed OpenSTA for the Liberty syntax and design-level rows.	U4, U6, U7
Views and round trip	Liberty view check, response writer, round trip through map.sh and ABC `cec`, three broken cases.	Asked for the handout to be re-read first and AGENTS.md to be brought up to date before any code.	U8
Runs and run time	Parallel targets, batched transistor-level simulation, the full runs and their reports.	Set the limits ("only what's absolutely necessary"); stopped one slow cell from holding up the final run.	U13, U14, U20, U21, U23
Sizing families	The `opt` family: an Elmore delay model fitted to inv_1, internal stages sized for the worst path.	Chose to build it ("Do option 2"), then had it reported as measured, not as predicted.	U19, U23
Portability and demo	Fixed `make check` under WSL and the stock-image build, rewrote `make demo` into its own folder.	Asked for the teammate's portability commit to be verified before relying on it.	U24, U25
Walkthrough GUI (branch `gui-walkthrough`)	The Cellforge page and its server.	Chose the style; asked for real check results and for stored cells to be loaded.	U22 (not yet committed)

Part	What the AI did	What the group did	Log
Who owned which part	Youssef Ayman Hassan: the synthesizer and some of the layout optimizations. Hassan Ashraf: the Euler paths and the routing. Yara Abdelkader: the characterizer, the request reader and the testing.		—

2. Tools, models and cost

Tool	Model	Used for	Cost
Claude Code (CLI and desktop app)	Claude Opus 5.5	All 23 logged sessions on `main`, 30 October to 11 November 2026, and the GUI	20 USD

No model is called by the flow. The only model output in the repository is code and documentation that the checks below had to accept.

3. How we described tasks, and how we checked the output

Describing tasks

- **One contract, written first.** `AGENTS.md` was in the first commit (`7e40e2e`, 25 October), before any code, and was extended on 30 October. It states how the project is checked (the exit status of each `make` target), the rules of overview section 4.3, what the assistant must never do (weaken a check, type in a rule value, move a tag, commit without being asked), and the conventions. The assistant reads it at the start of every session.
- **Goals, not code.** Requests named the outcome and the limit, in our words, which the log quotes: “minimize the length of the path that uses li1 if it is going to connect to met1 anyway” (U11), “Do not run the whole flow, only what’s essential” (U10), “I don’t want you to re-run the whole flow after improving each weakness” (U16).
- **Decisions stay with us.** Where a choice changed what the tool does (the synthesis objective, our Table 6 function, routing folded fingers on met1 against Tables 1 and 3 of the handout), the assistant asked and the member answered (U7, U10, U12). The met1 option was kept as a switchable policy, off by default.
- **Handout first.** Twice the assistant was told to re-read the handout before working (U7, U8); in U7 this found two missing items (M11, M13).

Checking the output

- **The check scripts decide.** A change was accepted only when the targets it touches exited 0: switch level, DRC in 24 tapped-row contexts, the layer check, LVS, RC-LVS, LVS against HD, the transistor-level and model-level checks, Table 5, the Liberty view check and the round trip. Each log row names the commands run and what was not run.
- **Before and after.** Optimizations were judged against the previous run, cell by cell (widths, vias, extracted R and C, delay). This is how M28 was found: both layouts were legal, only the comparison showed the loss.
- **Looking at the cells.** We reviewed the layout pictures ourselves. Two faults no check covers were caught this way: a pair of redundant contacts (M21) and met1 crowding the pins (M25).

One session from request to accepted change (U21)

Step	What happened
Request	“Let’s speed-up the transistor-level check. It’s insane if it’s 4 hours.” The check is a check script, so this was an explicit rule-9 request; the log row says so.
Limit set by the contract	AGENTS.md allows a check script to change only to become stricter, more general or faster with the same verdicts. So the cases, the transient timing, the thresholds and the verdict code had to stay as they were.
Change	Every DC row and every edge became its own copy of the cell in one netlist, run in a few ngspice decks instead of one ngspice run per case.
Evidence it is not weaker	Case-by-case comparison with the reports of the earlier run on the same netlists: identical verdicts, DC voltages and worst glitches for nand2_1, a22oi_1 and xor3_1. The broken case `nand2_1_pex_missing_pullup` was still rejected, on the same DC row and the same three edges. A missing model file still made the check fail. The original script was kept aside for the comparison.

Step	What happened
Result	20–28 s per cell. The change was then used in the final three-family run, where the transistor-level check passed on all 33 cells.

4. How we applied the rules of section 4.3

Rule	How it is applied
1. Pass or fail from check scripts	Every check exits non-zero when it fails and when it cannot run (AGENTS.md rule 5); `validate_characterizer.py` reports a Table 5 part that did not run as NOT RUN and fails (U6). The response's status is "pass" only when every check passed.
2. Numbers from files; rules from the PDK	Design-rule values live only in `src/rules.json`, each with its rule name, and `pdk_rules.py` proves every value against the installed `sky130A.tech` in `make check`. Frame numbers come from `given/plan_hd.json` only. The README and technical results were rewritten from `results/` after each full run (U14, U23); numbers from a scratch script were labelled as such.
3. A broken case for every check	18 cases in `tests/broken/` (listed in its README), plus cases derived at run time: a broken DRC context in every DRC run, 4 layer-removed copies of every cell (132 in the final run), a wrong-row Verilog model per cell. `make test` fails if any is accepted.
4. AI does not change checks, broken cases or the PDK	A check script changed only when a member asked for it in the session, and only to make it stricter, more general or faster with the same verdicts; each change is in the log (U1, U2, U6, U7, U8, U13, U21). Example: the transistor-level check was batched into one ngspice run per job (U21); the verdicts were compared case by case with the earlier reports, and the broken case was still rejected on the same row and edges.

5. The five most instructive AI mistakes

The mistakes log has 31 rows. For 20 of them, the log says a weaker check would have missed the fault; 11 were stopped by a check that would have stopped them in any form. These five taught us the most.

ID	What was wrong	How we caught it	Would a weaker check have missed it?
M5	The LVS wrapper started Netgen from a path the course image does not have, so every LVS run failed with exit 127.	`make lvs`: all 18 cells "netgen exited with 127".	**Yes.** The LVS broken case was still "rejected", only because Netgen never started. A broken case must fail *for the right reason*; after the fix, both LVS broken cases were confirmed to fail on the width mismatch and the invalid pin map they contain (U6).
M14	The decomposition renamed one subfunction's internal nets onto the other's, so two stages drove one net in xor3_1.	The decomposition tree picture showed two stages with one name.	**Yes.** The built-in check evaluated the stages in order, so the function was right. It checked the function, not the connectivity; `synthesize.py` now refuses two stages on one net.
M18	The router took a narrower route with met1 before a wider li1-only one, against Table 1's "met1 only where li1 cannot route".	`make compare`: the Table 4 wiring row failed for a21o_1 and mux2_1.	**Yes.** DRC, LVS and every simulation passed; only the comparison with HD checks which layers are used.
M27	A benchmark meant to plan regenerated specs planned the old ones: a patch never applied and its error went unread. The documentation then credited the wrong change with two improvements.	Reading the benchmark's output: 27 cells planned, not the 8 asked for.	**Yes.** Every planned layout was legal, so no check could see that the wrong input had been planned. Only reading the evidence did.

ID	What was wrong	How we caught it	Would a weaker check have missed it?
M29	The new sizing used 0.36 um as the smallest transistor. sky130 extracts anything under 0.42 um as a different, special device.	LVS: the device classes differed on two cells; Netgen named the special FETs.	**No.** LVS caught it. The value had been taken from the wrong rule; it is now a rule in `rules.json`, proven against the PDK like the others.

How all 31 were caught

Our reading of the log's "How you caught it" column, with every row placed once:

Caught by	Rows	IDs, and what this tells us
A check or tool that failed (non-zero exit, error, refused data)	16	M2, M3, M5, M7, M15–M20, M23, M24, M26, M29–M31. About half the mistakes never got past a check. DRC, LVS and the Table 4 comparison caught different ones: none of the three would have caught them all.
Reading code, the handout or a run's output, with no check failing	12	M1, M4, M6, M8–M13, M22, M27, M28. These are the expensive ones: the checks passed, so only someone reading closely found them. Several came from reviewing one commit (M6, M8–M10) or from re-reading the handout (M11, M13).
Looking at a picture of the cell or network	3	M14, M21, M25. Two of these were caught by a member, not by the assistant. No check covers redundant contacts or met1 crowding the pins.

Two process mistakes from the last day belong with these. A command chain assumed that `make views` runs characterization; it does not, and the Liberty view check refused the stale data (M30). A dry run (`make -n`) still executes recipe lines that contain `$(MAKE)`, and one overwrote the stress-test log; the Liberty view check refused that too (M31). In both, a check that refuses missing or stale inputs kept a wrong result out of the deliverables.

6. One decision we would never hand to an AI

We would never hand an AI the design-rule numbers, or the decision of what counts as passing. A rule value that is wrong but legal-looking produces layouts that every downstream step accepts. M29 is exactly this: a value taken from the wrong rule passed planning and drawing, and only LVS noticed. The check that catches an AI getting it wrong is `pdk_rules.py` in `make check`, which proves every value in `rules.json` against the installed `sky130A.tech` and rejects the broken case `rules_wrong_poly_space.json` (0.20 um instead of 0.21); and AGENTS.md forbids a rule value anywhere else.

7. Where AI saved time and where it cost time

Saved

- **Large algorithmic code.** The exact Euler search, the routers, the dynamic programs and the characterizer are long, exacting programs that the assistant wrote and then debugged against the checks.
- **Run time it removed.** The transistor-level step took about 4 hours in the 10 November run (with 16 runs at once, swapping, M24); batched, it takes 20–28 s per cell with identical verdicts (U21). Planning nor3_stack_1 went from 1226 s to 1 s once its fingers shared poly (U10).
- **Reviews.** Reviewing a teammate's commit found six faults (M5–M10) before they reached a checkpoint, including the hidden LVS failure.

Cost

- **Untested assumptions about the machine.** Parallel defaults chosen from the core count, not memory: the full run swapped for 4 hours and two characterizations were killed out of memory (M24).
- **Work built, then dropped.** The pin patches were built, sampled and removed again (U18). The `opt` sizing took a day; characterization then showed it faster only on maj4 (−5.3 %) and slower on and2 (+34.4 %) and maj3 (+19.8 %) (U23). It is kept as a measured negative result.
- **Reruns after process slips.** M27 (the benchmark), M28 (a cap that regressed two cells), M30 and M31 each cost a rerun.

8. Next time

- **We would hand to an AI:** writing and refactoring code against a fixed check, reviewing commits against the handout, and running and summarizing long flows, because every one of these ends in an exit status we can see.
- **We would not:** design-rule values, the choice of what to compare and what counts as passing, and claims of improvement before they are measured (the `opt` sizing). Also not run-time limits (memory, parallelism) without measuring them first (M24).
- **We would change:** add the `AI-assisted` trailer from the first commit, fill the log's Commit column at each commit.